/Pilares Fundamentais De POO/

Flora Belas

/Quais são os pilares?/

- Herança
- Polimorfismo
- Encapsulamento
- Abstração

/Pilares/

Cada pilar responde a um problema diferente.

Pilar	Responde
Abstração	o que mostrar e o que esconder
Encapsulamento	quem pode mexer no quê
Herança	como não repetir código
Polimorfismo	Mesma ordem, respostas diferentes

/Herança/

- A **herança em Java** é um mecanismo da Programação Orientada a Objetos (POO) que permite criar uma nova classe (chamada de subclasse ou classe filha) aproveitando os atributos e métodos de uma classe já existente (chamada de superclasse ou classe pai).

/Herança/

Herança é quando uma classe aproveita tudo que outra já tem, sem copiar código.

/Herança/

- **Reutilização de código:** Evita a repetição de lógica, permitindo que a classe filha herde comportamentos comuns do pai.
- **Palavra-chave extends:** É usada na declaração da classe para indicar de qual classe ela está herdando. Exemplo: public class Cachorro extends Animal.
- **Relação "É um":** Define que a subclasse é um tipo específico da superclasse (ex.: um Cachorro **é um** Animal).
- **Palavra-chave super:** Usada para referenciar e chamar construtores ou métodos da superclasse.
- **Sobrescrita de métodos (*Override*):** A subclasse pode alterar a implementação de um método herdado da superclasse para se ajustar ao seu próprio comportamento.
- **Classe Object:** Todas as classes em Java descendem, direta ou indiretamente, da classe Object como raiz da hierarquia.
- A mãe não sabe que tem filha. **Só a filha declara.**

/Superclasse e Subclasse/

Superclasse é a mãe. Subclasse é a filha.
São os nomes técnicos. Vocês vão ver nos erros do compilador e na internet.
Superclasse = classe pai, classe mãe, classe base
Subclasse = classe filha, classe derivada. A que herda.
O “super” não quer dizer melhor. Quer dizer **acima** na árvore.
Por isso a palavra chave pra chamar a superclasse é super.

/extends/

- extends é a **palavra chave** que liga a filha à mãe.
- Vai na declaração da classe filha, depois do nome dela.
- Uma classe pode ter **uma mãe só**. (Mas pode implementar quantas interfaces quiser)
- Ao escrever extends, a classe filha recebe acesso a **TODOS** os atributos e métodos da classe mãe.
- protected na mãe = fechado pra fora, **aberto pras filhas**.
- Privado nem a subclasse vê.

/Sobrescrita/

- **Sobrescrever é a filha trocar um método que a mãe já tinha, por uma versão dela.**
- O método da filha tem **o mesmo nome, os mesmos parâmetros e o mesmo retorno** da mãe.
- A mãe continua com o método dela intacto. Quem trocou foi só a filha.
- @Override é a anotação que pede pro compilador conferir se você realmente sobrescreveu algo.

/Polimorfismo/

Polimorfismo é dar **a mesma ordem** pra objetos diferentes, e cada um executar **do seu jeito**.
Ele acontece em classes que tem ligação uma com a outra, seja através de herança (extends) ou interface (implements) e dentro da mesma classe.

Polimorfismo acontece quando criamos métodos com o mesmo nome em classes ligadas através de herança, interface e métodos criados dentro da mesma classe.

/Exemplos/

- Sobrecarga de métodos em uma classe (Criar métodos com o mesmo nome com parâmetros diferentes).
- Uma classe filha dar um Override no método da mãe,
- Uma classe implementando os métodos que assinou com a interface.

/Polimorfismo estático vs Dinâmico/

- Ambos tem o mesmo nome, a diferença é nos parâmetros.
- Polimorfismo estático os métodos tem os mesmos parâmetros.
- Polimorfismo dinâmico muda os parâmetros.